



Project report

Advanced Web Programming Final Project

24/06/2025

Lappeenranta–Lahti University of Technology LUT

Author:

Minh Pham Xuan (Minh.Pham.Xuan@student.lut.fi)

Contents

Declaration of AI usage:	3
1. Technology Choices	4
1.1 Front-end	4
1.2 Back-end	4
1.3 Other libraries and packages	4
2 Proposed features	5
3 Limitations and design decisions	6
4 Installation instruction	8
5 User manual	9

Declaration of AI usage:

The author of this report and the related project, Minh Pham, used the following AI-tools during the preparation of the project and during the development of the program.

AI tool name: ChatGPT

Purpose of use

FOR THE REPORT: To improve the quality and reduce grammar error of the report.

FOR THE PROJECT:

To quickly have an idea about what to start with when unfamiliar with the concept and to discard wrong approaches by discussing it with AI (though sometimes it does provide something too nonsense or completely missed the point).

Help with finding errors in the project and add suggestions on how the project can be improved. And finally, to skip repeated tasks like writing translation. Json for other language manually or I have trash UI mostly same as Home UI, so I asked AI to write the UI part, and I checked the contents later, as you can see in my commits. And to quickly generate/improve my simple UI to test features.

Explanation of the use of the tool

FOR THE REPORT: I used it to remove grammatical errors and improve readability.

FOR THE PROJECT:

During the preparation phase, AI was used to find potential edge cases, relevant technologies or concepts that should be researched before implementation.

During development, AI was used to improve my UI with responsive design and to debug+fix code, at least with App.tsx configuration issue and especially with respond body mismatches. I also let it choose most fitting respond status to later help with debugging and catching errors for error prevention.

Other code sections unrelated to AI contain reference to the original source and some sections also mention usage of AI in the comments.

Responsibility

The author, Minh Pham, takes full responsibility for the content of this report and the related project and has reviewed and edited the content generated by the possible use of AI tools.

1. Technology Choices

This part lists the technologies that were chosen and reasons why they were included in the project. There are three sections in total. First section discusses essential front-end framework; second core backend technologies and finally other feature-specific libraries/packages are addressed in a table format.

1.1 Front-end

- **React (TypeScript)** is used as the front-end framework
- **Default CSS** framework for the application UI and responsive design requirement

1.2 Back-end

Node.js as JS runtime environment as part of course requirement

Express framework to work with Node.js to develop web and mobile application.

MongoDB + mongoose due to familiarity and interoperability with Node.js

1.3 Other libraries and packages

Here is the list of external libraries that has been used in the project, other than the one mentioned in section 1.1 and 1.2. Additionally, how and on what purpose are they used on is explained in the second column of the table.

Taulukko 1 External libraries and frameworks and their usage in the project

External libraries and frameworks	Usage
jsonwebtoken	For JWT :)
bcrypt	Used for password hashing
QuillJs	Custom text editor
i18n	Internationalization: Language translation in front-end

Mongoose	Object Data Modeling library to ease development that use MongoDB as database.
----------	--

2 Proposed features

Below here is the table for implemented features and point proposal.

Taulukko 2 List of implemented features

Features	Max points
Basic features + well written documentation	25
Utilization of a frontside framework: React	3
The document editor includes wysiwyg editor of some sort (e.g. https://quilljs.com/ , https://editorjs.io/ , https://draftjs.org/)	2
The drive shows besides the name of the document also the creation and last updated timestamp	1
User can sort the documents in drive based on the name, creation/last edited timestamp	1
Users are able to select a profile picture for themselves + the image is stored in the server	2
The application has dark and bright modes	1
Translation of the whole UI in two or more languages	2
Add pagination to the document listing	2
User can upload documents (it is enough to allow only images) to her drive	2
Add search functionality of some sort	2
User has an option to clone existing document	1
Documents are moved first into recycle bin and user can then clean the trashes when she sees fit	2
Total	46

3 Limitations and design decisions

This section discusses the known limitations of the application (that I at least noticed and remembered). Then it explains certain design decisions that would benefit from further clarification, including the reasoning behind those choices and any trade-offs of using such design. Generally, my reasoning is to create a minimal workable solution that matches the requirement.

Taulukko 3 List of limitations

Limitation	Detailed description
Lock mechanism could be unstable but at least not self-lock.	I only run the program in localhost, so I could not test lock mechanism properly.
No way to remove assigned permission	I didn't add buttons to remove certain permission from selected user
Minimal input validation	Only restriction for input is to not be empty.
Limited use of reusable React components	Code is mostly clustered into large files and could be further refactored
Can't remove user's profile image from local file system when changing image	

Taulukko 4 Design decision, their reasoning and trade-offs

Design decision	Reasoning	Trade-off
Filename not unique	No restriction on this so I just ignore it and for this project this is not important	We lose one unique identifier
DriveFile model address all possible file types, not separately	A unified data structure. Ease the data handling between client and server, as well as route design.	Large number of overhead/body contents. A workaround was required because certain file types have mandatory requirements that do not apply to other file types. Break OOP inheritance principle.
Only owner has permission to rename and remove files	Simplify soft deletion implementation.	It would be more logical if

No permission removal options	There is no need to do this in the requirement, so I just ignored it	Would be better to do this, we just have too many buttons now already.
Long access token duration and login as only method to generate new token + no session token and token auto refresh	When I decided to implement session token and token auto refresh there is already too many React components that depends on it and it is very risky to refactor due to limited time to do this project.	No token auto refresh. Could be nice to create an interceptor to refresh access token.
Editor automatically have view permission	It makes sense for me	
User's profile picture do not use DriveFile model	Simplify development	
simple locking mechanism: Other than lock and unlock, to prevent self-lock and stale lock, we set timeout for lock to reset and to extend lock duration and prevent lock time-out, renew lock every 5 min	My only focus is to not self-lock document when something unexpected happens and I also got help on this from my friend and this is his implementation. I only slightly modified the details fitting to my program.	No concurrent access.
Using isSoftDeleted flag to implement.	I can only think about this approach when trying to develop MOVE TO TRASH feature	
Button for copying public link	Easy for user/developer to test public file link	

4 Installation instruction

Instruction to install and use the web applications are written below step by step

Clone the GITHUB repository to local machine.

Go to the folder you want the git repo to reside in then run this command:

git clone <https://github.com/xuanminhcpvn/Introduction-to-Web-Programming-Project.git>

Go to cloned folder:

cd CT30A3204-Advanced-Web-Applications-Final-Project

Run following npm commands in order

npm run install

npm run build => build dist

npm run start => to start server and client

Finally:

go to browser => type localhost:1234

5 User manual

When user first accesses the page, they will see the message in Figure 1, since they are not logged in.

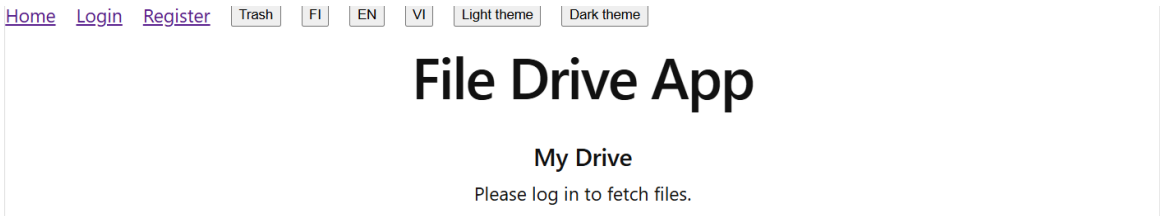


Figure 1 Home page if not logged in

Users can login by pressing the login link that can be seen in Figure 2.

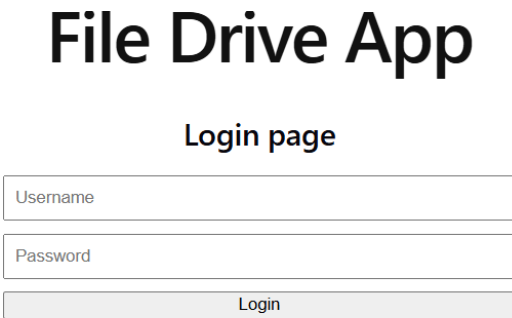


Figure 2 Login page

Since user probably don't have credentials yet, they are likely redirected to registration page that look like the one in Figure 3.

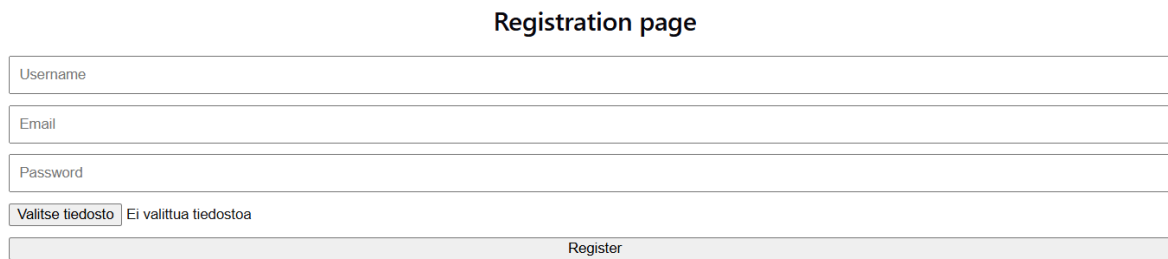


Figure 3 Registration page

After registration and login (which user is automatically redirected to after registration), they will see Home view for authenticated user. In Figure 4 below, I already created some files to show possible buttons and their features. When users are logged in, they can always go back to this page by clicking on the HOME link on top bar.

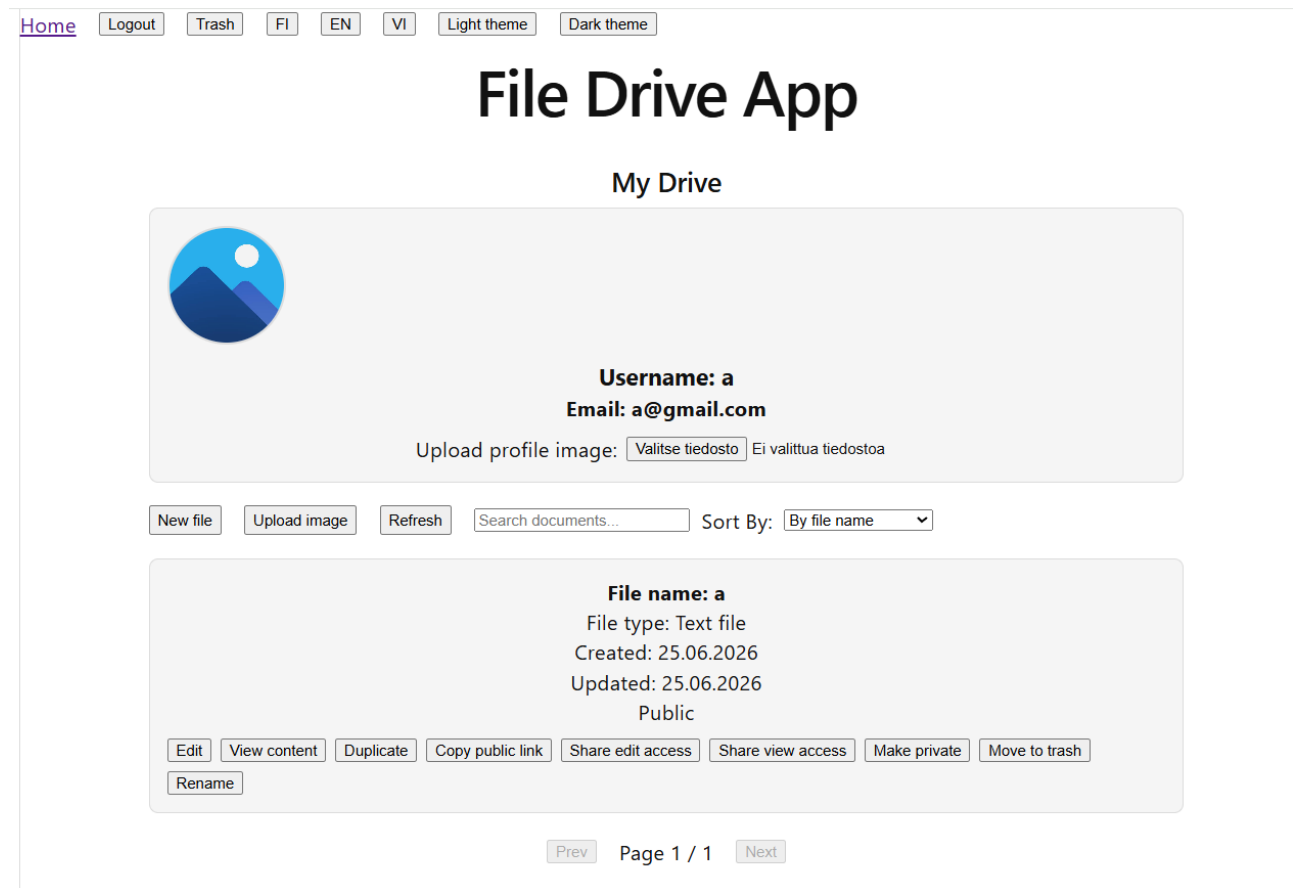


Figure 4 Authenticated Home Page contents

Edit button will open DocumentEdit page like the Figure 5 that has wysiwyg editor Quilljs to write contents. This is not available for image files. We have autosave feature in Editor, as well as manual "Save changes" button. View button is similar but view only. Public page is almost identical to View page. It is accessible by pasting the copied public link.

File Drive App

Tiedostonimi:

Normal ▾

B

I

U









lorem ipsum **bolded** underlined

Figure 5 Editor page

Trash button in top bar redirect to Trash page in Figure 6.

Move to trash button, move files there that can be restored to Home or delete from database permanently.

File Drive App

My Trash Bin

Sort By:

icon.jpg
Deleted at: 25.06.2026

a
Deleted at: 25.06.2026

Page 1 / 1

Figure 6 Trash page

Current theme is light theme Figure is Dark theme. Change theme by pressing button in top bar.

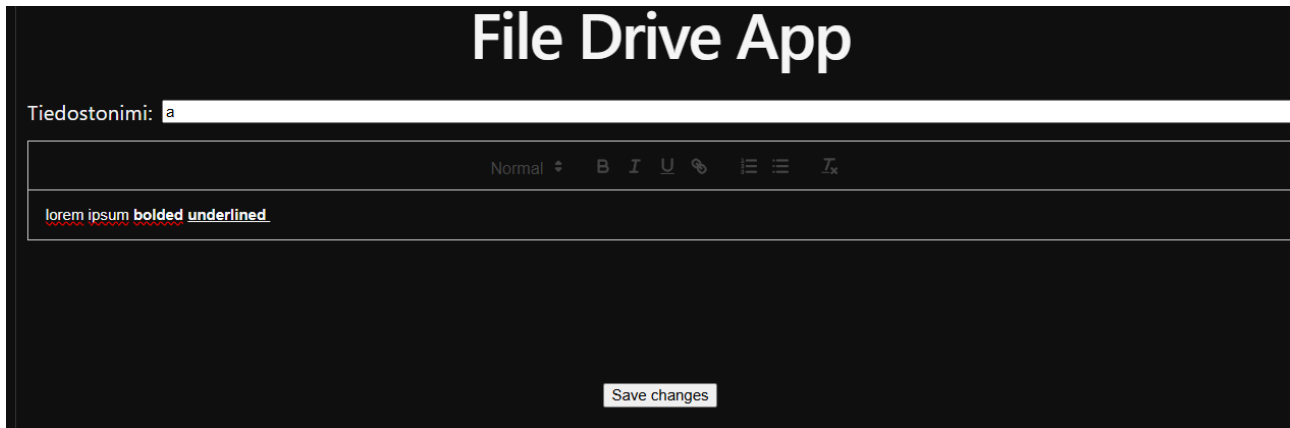


Figure 7 Dark mode

Logout let you see the view in Figure 1. Copy public link paste the link to the clipboard.

Most features that require certain input such as Share buttons, Rename and New File will make a prompt like in to read input. Errors that can be printed to user is also prompted to him/her. See Figure 9. Search feature, sort and pagination features are demonstrated in Figure 10.

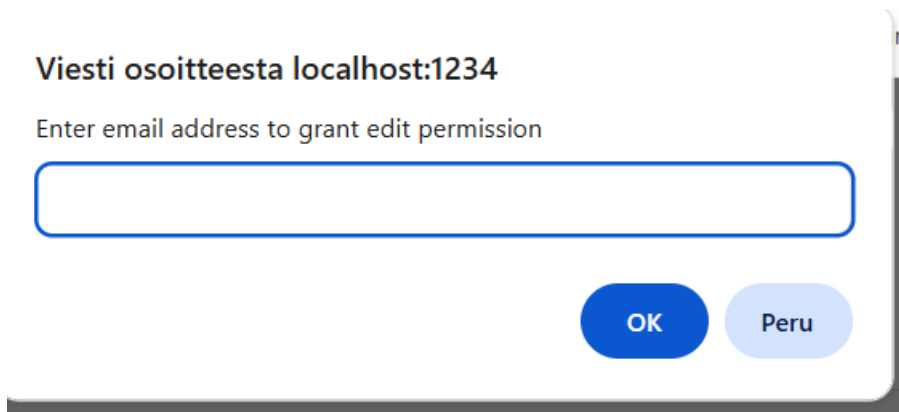


Figure 8 Prompt input from user

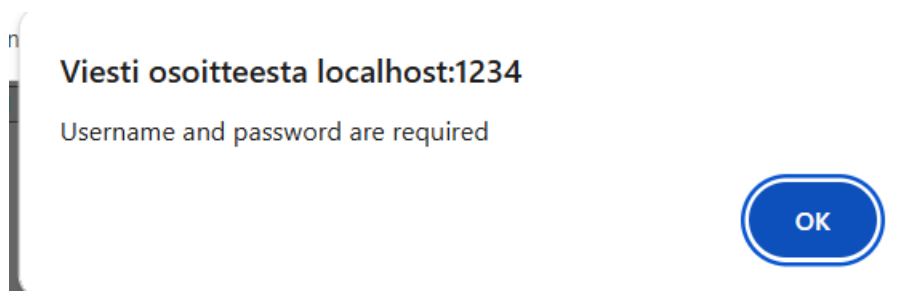


Figure 9 Error prompt

New file

Upload image

Refresh

Sort By:

By created date

By file name

By created date

Last updated date

File name: ab

File type: Text file

Created: 25.06.2026

Updated: 25.06.2026

Private

Edit

View content

Duplicate

Share edit access

Share view access

Make public

Move to trash

Rename

File name: a

File type: Text file

Created: 25.06.2026

Updated: 25.06.2026

Public

Edit

View content

Duplicate

Copy public link

Share edit access

Share view access

Make private

Move to trash

Rename

Prev

Page 1 / 1

Next

Figure 10 Search, sort pagination demonstration visually